

บทที่ 2

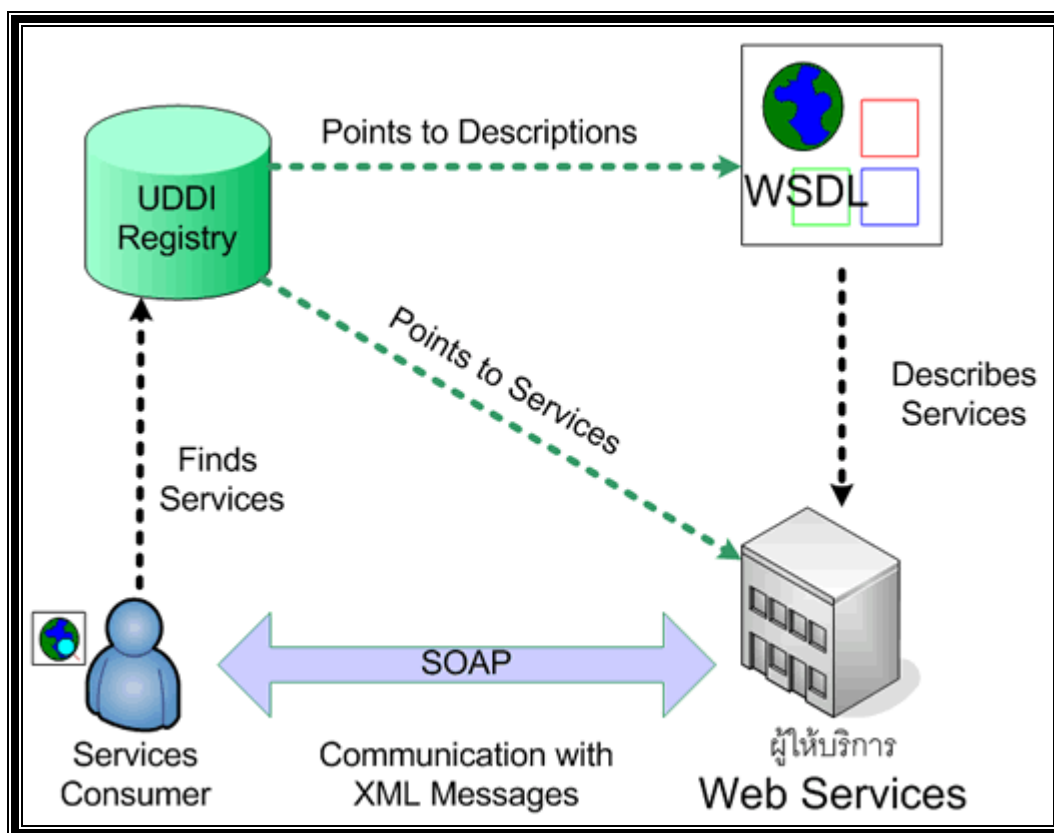
ทฤษฎีพื้นฐานที่ใช้ในโครงการ

2.1.เว็บเซอร์วิส

เว็บเซอร์วิสเป็นระบบซอฟต์แวร์ ที่ออกแบบมาเพื่อสนับสนุนการทำงานระหว่างคอมพิวเตอร์กับคอมพิวเตอร์ผ่านระบบเครือข่าย โดยที่ภาษาที่ใช้ในการติดต่อสื่อสารระหว่างคอมพิวเตอร์คือภาษาเอ็กซ์เอ็มแอล(XML) การอธิบายวิธีการใช้งานของเว็บเซอร์วิสนั้นจะอธิบายโดยใช้ภาษาวิสเดิล WSDL (Web Services Description Language) ซึ่งเป็นภาษา XML ประเภทหนึ่ง ระบบอื่นๆ จะสามารถติดต่อและทำงานกับเว็บเซอร์วิส โดยใช้โปรโตคอลที่ชื่อว่า SOAP (Simple Object Protocol) ซึ่งใช้ภาษา XML เป็นมาตรฐานในการติดต่อระหว่างระบบโดยผ่านทางโปรโตคอลอื่นที่ใช้ในการส่งข้อมูลบนเว็บ อาทิ เช่น โปรโตคอล HTTP

Webservice กับ Web Application ต่างกันอย่างไร

สิ่งที่ทำให้ต่างกันนั้นก็คือ จุดกำเนิดและจุดประสงค์ของทั้งสองนั้นต่างกัน Webservice นั้นเกิดมาจากการที่ Web Application ถูกพัฒนาได้จากหลายภาษา เช่น asp php jsp perl ทำให้การที่จะนำมารวมแล้วทำงานร่วมกันนั้นเป็นเรื่องที่ยากลำบาก (เหมือนคุยกันคนละภาษา) Webservice จึงเหมือนกับภาษาสากล ที่ทำให้แต่ละ Web Application ทำงานร่วมกันได้โดยผ่านทาง SOAP ที่มีรูปแบบเป็น XML



รูปที่ 2.1 Web service technology

Services Consumer:

เป็นใครก็ตามที่ต้องการเรียกใช้บริการจากผู้ให้บริการซึ่งสามารถค้นบริการที่ต้องการได้จาก UDDI Registry หรือ Service Registry หรือติดต่อจาก Provider โดยตรง

UDDI Registry:

ทำหน้าที่เป็นตัวกลางให้ผู้ให้บริการมาลงทะเบียนไว้ โดยใช้ WSDL ไฟล์ บอกรายละเอียดของ บริษัทและบริการที่มีให้ซึ่งอาจจะใช้หรือไม่ใช้ก็ได้

Provider:

เป็นผู้ให้บริการ มีหน้าที่ในการเปิดบริการเพื่อรองรับการขอใช้บริการจากผู้ใช้บริการ (Requestor) ที่เรียกเข้ามาขอใช้

2.2. XML (The Extensible Markup Language)

XML (The Extensible Markup Language 1.0) เป็นภาษา Markup ที่เป็น text-based ซึ่งทำให้เป็น มาตรฐานในการแลกเปลี่ยนข้อมูลบนอินเทอร์เน็ตอย่างรวดเร็ว ผู้ที่ทำหน้าที่รับผิดชอบ และ กำหนดมาตรฐานของ XML คือ World Wide Web Consortium (W3C) ความแตกต่างระหว่าง XML กับ HTML คือ HTML ถูกนำมาใช้ในการสร้าง เว็บเพจ ที่สามารถแสดงผลได้โดยโปรแกรมเบราว์เซอร์ แต่ XML จะใส่ tags ใ้ได้อย่างอิสระ แล้วทำการส่ง XML ชุดนี้ไป ประมวลผลยังแอปพลิเคชันใด ๆ ที่ สามารถใช้ข้อมูลใน XML นี้

2.2.1. XML and HTML

HTML ภาษาที่ใช้ในการเขียน Web มากที่สุดนั้นเป็นเพราะมีรูปแบบที่ง่ายต่อการแสดงผลของ Browser เนื่องจาก มี tag ตายตัวที่สามารถบอกได้ว่าเมื่อเจอ tag นี้จะแสดงผลอย่างไร เช่น เมื่อเจอ tag `<B>...</B>` ในเอกสารก็ให้แสดงข้อความที่อยู่ระหว่าง tag เป็นตัวหนา แต่จะสังเกตได้ว่าคอมพิวเตอร์ จะไม่เข้าใจว่าข้อความนั้นคืออะไร เพียงแต่รู้ว่าแสดงผลอย่างไร นั่นแสดงว่าไม่สามารถนำข้อมูล ภายใน tag เหล่านี้ไปทำการประมวลใดๆ ได้เลย

XML เป็นภาษาที่มีลักษณะเป็น tag คล้าย HTML แต่ไม่ได้มุ่งที่การแสดงผล XML มุ่งที่การสื่อ ความหมายโดยอนุญาตให้ผู้ใช้สามารถกำหนด tag ขึ้นได้เองเพื่อให้สื่อความหมายทางภาษาของมนุษย์ แต่คอมพิวเตอร์เองก็เข้าใจเช่นกัน ทำให้ข้อมูลระหว่าง tag สามารถนำไปประมวลผลต่อได้

2.3. SOAP (Simple Object Access Protocol)

SOAP กลายเป็นสิ่งที่มีความสำคัญสำหรับ Web Services อย่างรวดเร็ว เป็นโปรโตคอลที่ผู้ จัดหา Web Services เลือกใช้ที่จะส่ง message ระหว่าง Web Services SOAP เป็น Transport Protocol ที่ มี XML เป็นพื้นฐานและใช้ HTTP เป็นโปรโตคอลร่วมในการส่งผ่านเครือข่าย SOAP จะระบุวิธีในการ เข้ารหัสส่วนหัว (Header Encoding) ของทั้ง HTTP และไฟล์ XML ใ้อย่างชัดเจนทั้งใน ส่วนของการ ติดต่อไปยังคอมพิวเตอร์เครื่องหนึ่งและส่งผ่านข้อมูลไปให้ รวมถึงระบุวิธีที่โปรแกรมซึ่งถูกเรียกนั้น จะส่งค่าคืนกลับมาด้วย

SOAP (Simple Object Access Protocol) เป็น XML-based โปรโตคอล (lightweight protocol) และใช้ HTTP เป็นโปรโตคอลร่วม สำหรับ การแลกเปลี่ยนข้อมูลในสภาวะแวดล้อมแบบ

กระจายศูนย์ (decentralized, distributed environment) SOAP ได้ กำหนดเมสจิงโปรโตคอล (Messaging Protocol) ระหว่างผู้ขอบริการ (requestor) กับผู้ให้บริการ (provider) เช่น ผู้ขอบริการ สามารถติดต่อแลกเปลี่ยนข้อมูลกับผู้ให้บริการโดยใช้ RMI (Remote Method Invocation) ตามวิธีการของ โปรแกรมแบบออปเจ็ค บริษัทไมโครซอฟท์, ไอบีเอ็ม, โลตัส, ยูสเซอร์แลนด์ (User Land) และ ดีเวลลอปเปอร์เมนเตอร์ (Developer enter) ได้ร่วมกันกำหนดมาตรฐานของ SOAP ขึ้น ซึ่งต่อมาได้มีบริษัทอีก 30 กว่าบริษัทเข้าร่วมและ จัดตั้งเป็น W3C XML Protocol Workgroup ขึ้น SOAP ได้กำหนดรูปแบบพื้นฐานของการสื่อสารแบบกระจายขึ้นโดย การพัฒนา SOA แม้ว่า SOA จะไม่ได้กำหนดเมสจิงโปรโตคอล (Messaging Protocol) ไว้ แต่ SOAP ได้ถูกกำหนด ให้เป็น Services-Oriented Architecture Protocol เรียบร้อยแล้ว เนื่องจากมันได้ถูกใช้ในการพัฒนา SOAP อย่างแพร่ หลายแล้วนั่นเอง จุดเด่นของ SOAP ก็คือเป็นโปรโตคอลที่เป็นกลาง กล่าวคือ ไม่มีใครเป็นเจ้าของและเป็นโปรโตคอล ที่ทำงานกับโปรโตคอลอื่นหลายชนิด การพัฒนาก่อนุญาตให้ทำได้อย่างอิสระตามแพลตฟอร์มระบบปฏิบัติการ แบบจำลองทางวัตถุ (Object model) และภาษาโปรแกรมของผู้ที่ทำการพัฒนา

2.4. WSDL (Web Services Description Language)

WSDL (Web Services Description Language) เป็นภาษาที่ใช้อธิบายคุณลักษณะการใช้บริการของ Web Services และวิธีการติดต่อกับ Web Services ความต้องการของนิยามนี้เกี่ยวเนื่องกับความต้องการของ distributed system ที่จะกำหนด Interface Definition Language(IDL) โดยใช้ภาษา XML, WSDL เกิดจากการรวมแนวคิดของ NASSL (The Network Accessible Service Specification Language), WDS (Well-Defined Services) ของบริษัทไอบีเอ็ม, SDL (The Service Description Language) และ SCL (the SOAP Contract Language) ของบริษัทไมโครซอฟท์ ปัจจุบัน WSDL เป็นภาษา ที่อยู่ในการดูแลของ W3C (World Wide Web Consortium) ซึ่งยังไม่เป็นมาตรฐานที่สมบูรณ์ เวอร์ชันที่ใช้งานอยู่ใน ปัจจุบันคือ WSDL 1.1 (รายละเอียดเพิ่มเติมเกี่ยวกับ WSDL สามารถศึกษาเพิ่มเติมได้ที่ <http://www.w3c.org/TR/wsdl>)

WSDL คือ มาตรฐานสำหรับการประกาศ process ที่จำเป็นในการเรียกใช้เซอร์วิส SOAP (Simple Object Access Potocal)

2.4.1. โครงสร้างเอกสาร WSDL

WSDL เป็นภาษาที่อยู่ในความดูแลขององค์กร W3C (World Wide Web Consortium) version ที่มีอยู่ในปัจจุบัน คือ WSDL 1.1 ในการใช้งานจริง หากเราสร้างบริการ Web Services ก็จะมีเครื่องมือช่วยสร้างเอกสาร WSDL สำหรับ Web Services อย่างอัตโนมัติ จุดภายในเอกสารที่เราควรรู้เกี่ยวกับการติดต่อและเรียกใช้บริการของ Web Services มีจุดที่ควรรู้ ดังนี้

ตารางที่ 2.1แสดงโครงสร้างของ wsdl

Element	Definition
<portType>	เป็นส่วนที่สำคัญที่สุดในWSDL element อธิบาย operations ที่ web service มีให้บริการและ messages ที่เกี่ยวข้อง เทียบได้กับ function library หรือ module หรือ class ในการเขียนโปรแกรม
<operation>	อธิบาย method ที่ให้บริการ Web Services หนึ่งจะมี method จำนวนกี่ method ก็ได้
<message>	อธิบาย data elements ของ operation แต่ละ message อาจมีมากกว่าหนึ่งส่วนเทียบได้กับ parameter ของ function ในการเขียนโปรแกรม
<types>	อธิบายชนิดข้อมูลที่ web serviceใช้ เพื่อความเป็นกลาง WSDL ใช้ XML Schema syntax ในการระบุชนิดข้อมูล
<binding>	อธิบาย format ของ message และ protocol details ในแต่ละ port
<service>	สำหรับ web server จะมี Web Services จำนวนกี่บริการก็ได้ และ ชื่อ Web Services ก็เป็นตัวจำแนกและบ่งบอกแต่ละบริการซึ่งห้ามมีชื่อซ้ำกัน

ตามทฤษฎีแล้ว ไฟล์เอกสาร WSDL แต่ละไฟล์ สามารถอธิบายคุณลักษณะของบริการ Web Services ได้มากกว่า 1 บริการโดยแต่ละ Web Services จะมี port สื่อสารเฉพาะตัว ซึ่งบ่งบอกไว้ในเอกสาร WSDL อยู่แล้ว

2.5. UDDI (Universal Description, Discovery and Integration)

UDDI เป็นที่เก็บรวบรวม Web Services ต่างๆ ในอินเทอร์เน็ต ไว้ในแหล่งเดียวกันเพื่อให้ผู้ใช้บริการสามารถค้นหาได้ง่ายๆ หากเปรียบเทียบง่ายๆ ให้มองเหมือนสมุดหน้าเหลืองที่เราใช้ในการเปิดดูเบอร์โทรศัพท์

- ผู้เริ่มก่อตั้ง UDDI ในช่วงแรกคือ IBM และ Microsoft และ Ariba ซึ่งเป็นบริษัทที่ทำธุรกิจ B2B ปัจจุบันมีบริษัทที่มีส่วนร่วมในการกำหนดมาตรฐานของ UDDI มากกว่า 70 บริษัท
- UDDI ถูกสร้างขึ้นมาเพื่อเป็นมาตรฐานในการค้นหาบริการของ WS สำหรับคู่ค้าทางธุรกิจ

(Business Partner)

- UDDI Business Registry เป็นฐานข้อมูล WS ของบริษัทคู่ค้าทางธุรกิจ
- ในปัจจุบันบางบริษัทก็ตั้งตัวเองเป็นตัวแทนผู้ให้บริการ (Service brokers)

2.6. Microsoft .NET

คำว่า .NET มาจากโครงการของทางไมโครซอฟท์ที่ต้องการจะผนวกรวมการพัฒนาแอปพลิเคชันของทาง Desktop และทาง Web เข้าด้วยกัน ซึ่งผลิตภัณฑ์ในอนาคตทั้งหมดของไมโครซอฟท์จะทำงานอยู่บนพื้นฐานของ .NET ทั้งหมด พุดแบบง่าย ๆ คือ .NET นั้นถูกออกแบบมาโดยมีพื้นฐานของอินเทอร์เน็ตอยู่ในใจ (Highly distributed environment of the Internet) ทำให้อินเทอร์เน็ตเป็นพื้นฐานอยู่ภายในระบบปฏิบัติการ (Operating System) ซึ่งแนวคิดดังกล่าวนี้ได้รับแรงบันดาลใจมาจากหลาย ๆ สิ่งแต่สิ่งหนึ่งในนั้นมาจากภาษา Java โดยเฉพาะอย่างยิ่งในเรื่องของ Java Virtual Machine อาจกล่าวได้ว่า Microsoft .NET หรือเรียกสั้น ๆ ว่า .NET ก็คือแพลตฟอร์ม (สถานะแวดล้อมของฮาร์ดแวร์หรือซอฟต์แวร์ที่โปรแกรมใช้ในการรัน) ใหม่สำหรับการสร้างแอปพลิเคชันที่มีลักษณะตั้งแต่แบบ standalone (รันบนคอมพิวเตอร์เครื่องเดียว) จนถึงแบบ web-based application และแบบ web service โดย .NET ได้ให้ .NET Framework สำหรับสร้างแอปพลิเคชันต่าง ๆ โดยมี Visual Studio .NET เป็นเครื่องมือสำหรับสร้างแอปพลิเคชันต่าง ๆ ที่สนับสนุน .NET Framework ซึ่งจะมีหลายภาษาให้เลือกใช้ เช่น Visual C++ .NET, C# (ซีชาร์ป คือ ภาษาตัวใหม่จากค่ายไมโครซอฟท์ซึ่งคล้ายภาษา Java)

2.6.1 องค์ประกอบของ Microsoft.NET

2.6.1.1. ภาษาที่ใช้ในการพัฒนาแอปพลิเคชัน: Microsoft.Net เลือกใช้ภาษา C# เป็นหลัก ไมโครซอฟท์พัฒนาภาษา C# ขึ้นมาใหม่เพื่อใช้กับ .Net โดยเฉพาะ ภาษา C# มีรากฐานมาจากภาษา C และ C++ แอปพลิเคชันที่เขียนด้วย C# อาจเลือกคอมไพล์เป็นไบนารีโค้ดในฟอร์แมต Internal

Language (IL) ไบท์โค้ดที่ได้จะทำงานบน Common Language Runtime (CLR) นอกจากนี้ยังอาจเลือกคอมไพล์ C# เป็นเนทีฟโค้ดเลยก็ได้

2.6.1.2. คอมโพเนนต์พื้นฐาน : ไมโครซอฟท์เตรียมคอมโพเนนต์สำหรับเรียกใช้ใน Microsoft.Net ในชุด .Net Framework SDK

2.6.1.3. ไดนามิกเว็บเพจ: Microsoft.Net สามารถสร้างเว็บเพจแบบไดนามิกด้วย Active Server Page หรือ ASP.Net โดยอาจเลือกใช้วิชวลเบสิกหรือ C# มาเขียนโปรแกรม เพื่อคอมไพล์เป็นเนทีฟโค้ดเลยก็ได้

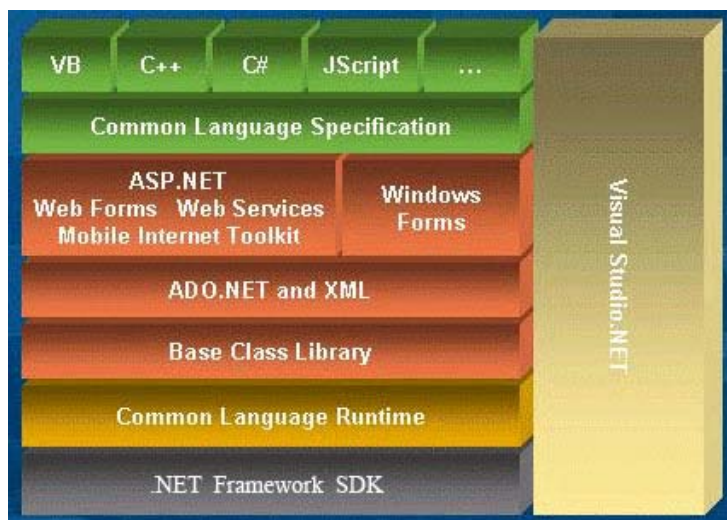
2.6.1.4. รันไทม์ไลบรารี: ไลบรารีพื้นฐานที่โปรแกรมโค้ดจะทำงานอยู่บนไลบรารีของ Microsoft.Net ใช้ Common Language Runtime เป็นพื้นฐานให้ไบท์โค้ดทำงาน จุดเด่นของ CLR ก็คือสนับสนุนการใช้ภาษาคอมพิวเตอร์อื่นๆ ผู้ใช้สามารถใช้ภาษา Visual Basic, Perl, Eiffel, COBOL, Fortran มาเขียนโปรแกรมได้ เพราะโปรแกรมที่เขียนจะคอมไพล์เป็นไบท์โค้ดเพื่อทำงานบน CLR

2.6.1.5. ยูสเซอร์อินเตอร์เฟซ: ใน Microsoft.Net มียูสเซอร์อินเตอร์เฟซคอมโพเนนต์ให้ใช้คือ Win Forms และ Web Forms การใช้งานจะเรียกผ่าน Microsoft Visual Studio.Net ซึ่งมี IDE ทั้งสองแบบที่สามารถนำมาออกแบบแอปพลิเคชันได้

2.6.1.6. ดาต้าเบสแอคเซส: การติดต่อกับดาต้าเบสบน Microsoft.Net นั้นมี ADO.Net ให้ใช้สามารถใช้งานร่วมกับ XML เพื่อนำดาต้าเบสมาใช้งานบนเว็บเพจ เว็บเซอร์วิสของ .Net จะใช้ประโยชน์จาก ADO มาก

2.6.2 สถาปัตยกรรม .NET

เราทราบแล้วว่า .NET เป็นนิยามการให้บริการของซอฟต์แวร์ในรูปแบบของเซอร์วิส ซึ่งจะรันได้โดยไม่ขึ้นอยู่กับอุปกรณ์ที่ใช้แสดงผล หรือระบบปฏิบัติการใดๆ ในส่วนนี้จะกล่าวถึงโครงสร้างโดยรวมทั้งหมดของการสร้างแอปพลิเคชัน .NET (.NET Application Architecture)ซึ่งแสดงดังรูปที่ 2.2



รูปที่ 2.2 โครงสร้างการพัฒนาแอปพลิเคชัน .NET

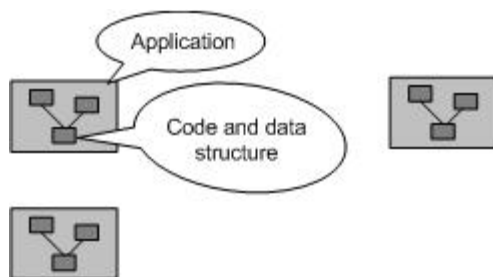
รูปที่ 3-1 แสดงถึงสถาปัตยกรรมของแอปพลิเคชัน .NET ที่พัฒนาด้วย Visual Studio.net ถือว่าเป็นการเปลี่ยนแปลงครั้งสำคัญ ทั้งแพลตฟอร์มที่ใช้ในการพัฒนาแอปพลิเคชันและสถาปัตยกรรมที่ใช้โดยมีเลเยอร์ล่างสุดคือ .NET Framework SDK เปรียบเสมือน Runtime Library ที่จะรันอยู่ คอยสนับสนุนการทำงานของแอปพลิเคชัน จากนั้นจะเป็นเลเยอร์ของ Common Language Runtime เป็นผลลัพธ์ของการคอมไพล์ (compile) แอปพลิเคชัน .NET เลเยอร์ถัดขึ้นมาเป็นเครื่องมือ (Tools) และเทคนิคต่างๆที่เราสามารถใช้พัฒนาแอปพลิเคชันได้ทั้งในเรื่องของ web service, ADO.net, ASP.net จะกระทั่งถึงเลเยอร์บนสุดคือ ภาษาที่ใช้ในการพัฒนาแอปพลิเคชันด้วย Visual Studio.net

2.6.2.1 เลเยอร์ .NET Framework SDK

สถาปัตยกรรม .NET Framework คือ กรอบการทำงานของการทำงานของการเขียนโปรแกรมที่ไม่โครงสร้างที่คิดขึ้นมาเพื่อรองรับการติดต่อสื่อสาร เพื่อแลกเปลี่ยนข้อมูล (Exchange Data) ระหว่างกัน หรือแลกเปลี่ยนข้อมูลระหว่างแพลตฟอร์ม (Platform) ให้มีความสมบูรณ์ยิ่งขึ้น โดยอาศัยภาษา XML (Extensible Markup Language) ทำหน้าที่เป็นตัวกลางในการแลกเปลี่ยนข้อมูลระหว่างแพลตฟอร์มไฟล์ของฐานข้อมูล

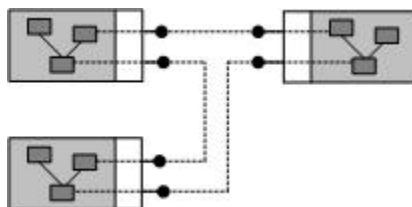
2.6.2.2 เลเยอร์ Common Language Runtime

ก่อนที่จะมีการพัฒนาโปรแกรมเป็น Object Oriented นั้นแอปพลิเคชันแต่ละตัวเปรียบเสมือนกล่อง ภายในแอปพลิเคชันก็จะมีโค้ด(code) มีโครงสร้างข้อมูล (data structure) ต่างๆของตัวเอง มีฟังก์ชันต่างๆของตัวแอปพลิเคชันนั้นๆ ดังแสดงในรูปที่ 2.3



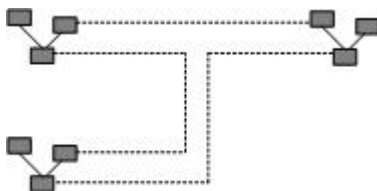
รูปที่ 2.3 โครงสร้างแอปพลิเคชันแรกๆ การติดต่อกันระหว่างแอปพลิเคชันเป็นเรื่องยาก

การที่แอปพลิเคชันต่างๆ จะมีการเรียกใช้การทำงานของกันและกัน หรือมีการส่งผ่านข้อมูลถึงกันและกัน เป็นสิ่งที่ทำได้ยาก ซึ่งอาจต้องมีการกำหนดอะไรขึ้นมาเองระหว่าง 2 แอปพลิเคชันนั้นๆ จนกระทั่งในยุคถัดมา ไมโครซอฟท์ได้คิดค้นเทคโนโลยี COM (Component Object Model) เป็นวิธีที่ทำให้เราเขียนโปรแกรมเป็นแบบ Object Oriented และเรียกใช้การทำงานที่มาจากต่างแอปพลิเคชันได้ ดังแสดงในรูปที่ 2.4



รูปที่ 2.4 แสดงการพัฒนาแอปพลิเคชันแบบ COM ที่สามารถเรียกใช้ฟังก์ชันของแอปพลิเคชันอื่นได้

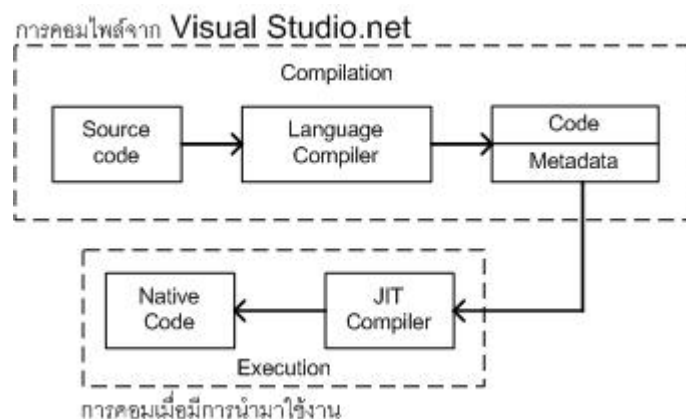
หากจะอธิบายให้ง่ายขึ้น ก็คือเปรียบเทียบเราเอาแพ็คเกจอันหนึ่งห่อแอปพลิเคชันของเราไว้ และการพูดคุยกันของแอปพลิเคชันก็พูดคุยผ่านแพ็คเกจที่เราห่อเอาไว้ จะกระทั่งมาถึงตัว Visual Studio.net ที่ได้รับการออกแบบใหม่ จะเห็นว่าจากรูปเดิม กล่องแพ็คเกจหายไป ถ้าเราพัฒนาด้วยรูปแบบเทคโนโลยี .NET นั้น คลาสต่างๆ สามารถติดต่อกันได้โดยตรง ดังรูปที่ 2.5



รูปที่ 2.5 การพัฒนาแอปพลิเคชันบนเทคโนโลยี .NET คลาสภายในแต่ละแอปพลิเคชันสามารถติดต่อกันได้โดยตรง

การพัฒนาแอปพลิเคชันด้วย Visual Studio.net นั้น เมื่อเราคอมไพล์สิ่งที่เราได้ จะไม่ใช่ไค้ดไบนารี (Binary Code) เลยทีเดียว แต่จะได้เป็นภาษากลางอันหนึ่งเรียกว่า Microsoft Intermediate

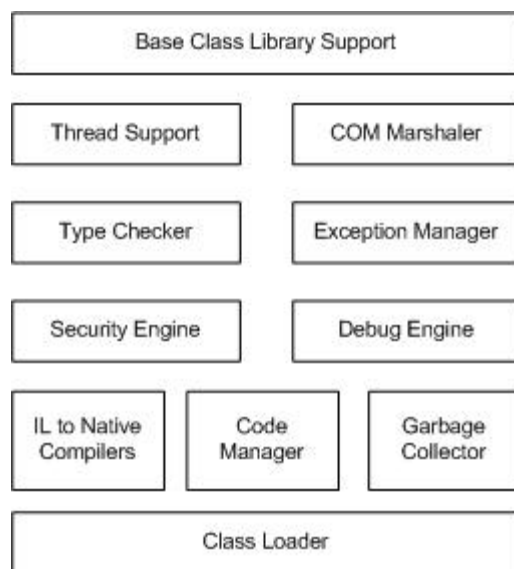
Language (MSIL) ซึ่งเป็นภาษาในระดับเลเยอร์ต่างๆ โครงสร้างของภาษา (Syntax) จะเหมือนภาษา Assembly ภายในสิ่งที่เกิดจากการคอมไพล์ก็จะเป็น MSIL ตัวนี้ ภายในตัวมันจะประกอบไปด้วย 2 ส่วน คือ โค้ด กับตัวแอตทริบิวต์หรือพรีพอร์เตอร์ต่างๆ ที่ใช้อธิบายตัวโค้ดนั้นซึ่งเรียกว่า Metadata ดังรูปที่ 2.6



รูปที่ 2.6 โครงสร้างการคอมไพล์จาก Visual Studio.net และเมื่อมีการนำไปใช้งานจริง

จากนั้น เมื่อโค้ดซึ่งเป็น Intermediate Language ถูกเรียกใช้งานจริงๆ จะมีตัว Compiler (ตัวแปลภาษา) ตัวหนึ่งมาทำการคอมไพล์โค้ดตัวนั้นให้เป็นโค้ดไบนารี ซึ่ง Compiler ตัวนั้นจะเรียกว่า Just In Time Compiler (JIT Compiler) ที่เรียกว่า Just In Time เพราะว่าจะมีการคอมไพล์เมื่อมีการใช้งาน ฉะนั้นคลาสหรือโค้ดต่างๆ ที่เราพัฒนาขึ้นแล้วจะถูกคอมไพล์มาเป็น Intermediate Language ที่มีโครงสร้างของภาษาแบบเดียวกัน เพราะฉะนั้นคลาสต่างๆ ในแอปพลิเคชันจึงสามารถทำงานได้อย่างกลมกลืนกันและไม่มีข้อติดขัดอะไร

การทำงานของ Common Language Runtime นั้นภายในตัว Common Language Runtime จะมีโมดูล (Module) ย่อยๆ ซึ่งเป็นสถาปัตยกรรมภายในดังรูปที่ 2.7



รูปที่ 2.7 แสดงสถาปัตยกรรมของ Common Language Runtime

ด้านล่างสุดในรูปที่ 3-6 จะมี Class Loader ซึ่งเอาไว้โหลดโปรแกรมของเราขึ้นมาทำงาน นอกจากนี้ก็จะมี Compiler ซึ่งทำการคอมไพล์ภาษา Intermediate Language ให้เป็นภาษาไบนารี โดยจะมีตัว Code Manager และ Garbage Collector คอยจัดการกับหน่วยความจำที่เราจองเวลาเรียกใช้งาน นอกจากนี้ก็จะมีเรื่องของระบบความปลอดภัย (Security) ในการทำงาน รวมทั้ง Debug Engine ในการดัก Runtime Error และตัว Exception Manager การตรวจเช็คชนิดของตัวแปรต่างๆ และด้านบนสุดจะเป็นการใช้งานร่วมกับ Library Class ต่างๆ ซึ่งจะเตรียมมาให้เพราะฉะนั้นเลเยอร์ของ Common Language Runtime การคอมไพล์แอปพลิเคชันใดก็ตาม ไม่ว่าจะเป็น ASP.net หรือเขียนแอปพลิเคชันบน Windows ธรรมดาหรือจะเป็นการเขียน Web service ก็ตาม สิ่งที่ได้จากการคอมไพล์จะเป็น Common Language Runtime

นั่นคือการออกแบบเพื่อสนับสนุนการทำงานร่วมกันใน Common Language Runtime จึงจัดข้อเสียของ COM ไปได้ เนื่องจากข้อจำกัดของ COM คือเป็นเพียงการเอาอะไรบางอย่างมาห่อคลาสเอาไว้เท่านั้น ดังนั้นเวลาที่เอา COM ไปใช้งานจึงค่อนข้างยุ่งยาก รวมทั้งถ้ามีการเปลี่ยนแปลง COM นั้นๆก็จะทำให้เกิดปัญหาเรื่องของการเข้ากันได้ (Compatibility) ระหว่างเวอร์ชันเดิมกับเวอร์ชันปัจจุบันแต่ถ้าเราพัฒนาด้วย Visual Studio.net ข้อเสียของ COM ทั้งหมดก็就会被ขจัดเสีย

ความจริง Common Language Runtime เป็นหลักการทำงานที่มีวิวัฒนาการมาจาก COM อีกทีหนึ่ง เป็นการทำ Object Oriented ที่แกนของภาษาเลย โดย Visual Studio.net นั้นถูกออกแบบเพื่อสนับสนุน Object Oriented โดยเฉพาะ คลาสต่างๆที่อยู่ในแต่ละแอปพลิเคชันสามารถติดต่อถึงกันได้โดยตรง (Inherit) ในแอปพลิเคชัน A เราอาจจะเขียนคลาสด้วยภาษา C++ และแอปพลิเคชัน A อาจจะ

ติดต่อกับแอปพลิเคชัน B ซึ่งเขียนด้วยภาษา Visual Basic (VB) ได้ คือการติดต่อกัน (Inherit) ข้ามภาษานั้นสามารถทำได้ดังแสดงในรูปที่ 2.8



รูปที่ 2.8 ความสามารถในการติดต่อข้ามแอปพลิเคชันเมื่อเขียนด้วยภาษาที่ต่างกันใน Visual Studio.net

ใน Visual Studio.net จะคอมไพล์เป็นภาษาเดียวกันคือ Intermediate Language ตามรูปก่อนหน้านี้ คือคอมไพล์เป็นภาษาอันหนึ่ง เพราะฉะนั้นจึงสามารถ Inherit กันข้ามภาษาได้ นอกจากนี้ยังสามารถทำงานด้วยกันกับ COM แบบเดิมที่เราเคยเขียนมาแล้วได้ด้วย ซึ่งใช้ Visual Studio เดิมโค้ดเหล่านี้ก็ไม่จำเป็นต้องโยนทิ้ง ใน Visual Studio.net เราสามารถเรียกใช้งาน COM ได้ตามปกติและในทางกลับกัน ใน Visual Studio 6.0 ที่มีอยู่ก็สามารถเรียกใช้งานคอมโพเนนต์ที่เขียนด้วย Visual Studio.net ได้เช่นกัน คือเป็นการเข้ากันได้ทั้งสองทาง (Backward-Forward Compatibility) นี่คือข้อดีมากมายของ .NET ทำให้เราไม่ต้องมาพัฒนาโค้ดใหม่

2.6.2.2.1 การจัดการกับหน่วยความจำเมื่อทำการประมวลผล

เนื่องจากการทำงานทั้งหมดของตัว .NET จะมีการดูแลในเรื่องของ Common Language Runtime ปัญหาหนึ่งที่มีจะพบ เมื่อเราพัฒนาแอปพลิเคชัน คือเรื่องของ Garbage Collection ในการจัดการกับหน่วยความจำ โดยเฉพาะบางภาษาอย่างเช่น C++ ที่ต้องมีการเรียกใช้งานพอยเตอร์ (Pointer) ค่อนข้างมาก ในการใช้งานพอยเตอร์นั้นถ้าเราเขียนผิดไปนิดเดียวก็อาจจะทำให้แอปพลิเคชันของเราแฉกในระหว่างรันหรือประมวลผลก็ได้ ทำให้เมื่อแอปพลิเคชันของเรารันไปเรื่อยๆ หน่วยความจำส่วนนี้ก็จะขยายไปเรื่อยๆ และสักพักหนึ่งก็จะเกิดการหยุดไป

การแก้ไขข้อผิดพลาด (Debug) ตรงนี้ทำได้ยากมาก เพราะว่ากว่าเราจะรู้ว่าเกิดความผิดพลาดขึ้นเราก็กว้างเวลาในการรันไปแล้ว ซึ่งการจัดการกับหน่วยความจำตรงนี้ ถ้าเราพัฒนาด้วยตัวแพลตฟอร์มของ .NET จะมีเครื่องมือตัวหนึ่งเรียกว่า Garbage Collector เป็นตัวที่คอยจัดการกับหน่วยความจำให้เรา ความจริงตัว Garbage Collector จะเป็นตัวที่ทำการกำหนดค่าหน่วยความจำให้ว่างให้เราเองในส่วนที่เราไม่ใช้งานโดยอัตโนมัติ นี่คือข้อดีอันหนึ่งในการพัฒนาด้วยแพลตฟอร์มของ .NET

2.6.2.2.2 ระบบการตรวจจับความผิดพลาด

นอกจากนี้ในเรื่องของ Exception Manager หรือการดัก Runtime Error ในการสร้างแอปพลิเคชันที่มีการรันได้อย่างดีนั้น การดัก Runtime Error ที่จะเกิดขึ้นในระหว่างทำงาน ก็เป็นปัจจัยหนึ่งที่เรจะต้องคำนึงถึง ซึ่งในตัว .NET จะมี Syntax ในการดัก Runtime Error ที่มีลักษณะเป็นโครงสร้างทำให้หาได้ง่ายขึ้น เราเรียกว่า Structure Exception Handling

2.6.2.2.3 รูปแบบการคอมไพล์แอปพลิเคชัน

นอกจากนี้การคอมไพล์แอปพลิเคชันด้วย .NET มีให้เลือกหลายแบบ บางคนอาจจะคอมไพล์ให้เป็นไค้ดไบนารี (Binary Code) เลยโดยไม่ต้องคอมไพล์ให้เป็น Intermediate Language ก็สามารถพัฒนาได้ โดยใช้ Visual C++ .net เรียกว่า Managed C++ หรือถ้าเราเลือกใช้เครื่องมือภาษาอื่นๆที่มีความง่ายในการพัฒนายิ่งกว่าคือ VB หรือ C# ก็สามารถคอมไพล์เป็น Intermediate Language ได้

ในการคอมไพล์จะไม่มีตัว Interpreter Runtime Library ต่างๆ เช่น VB ที่ต้องมี Runtime Library ของตัวเองเวลาพัฒนาด้วย Visual Studio 6.0 อันนี้ก็จะไม่จำเป็นต้องมีอีกต่อไป เราใช้ตัว Runtime อันเดียวกัน ก็คือตัว .net Framework เป็นตัว Runtime Library ที่กล่่าวมาข้างต้น

2.6.2.2.4 เรื่องของชนิดตัวแปร

ภาษาทุกตัวที่เขียนในตัว .NET จะเป็นภาษาแบบ Type-Safety คือการเปลี่ยนแปลงระหว่างค่าชนิดต่างๆ เป็น String type ตัว Compiler จะตรวจสอบได้ ถ้าเรามีการติดต่อระหว่างตัวแปร 2 ตัวที่มีชนิดต่างกันเช่น เราอาจต้องการเปลี่ยนชนิดของค่าในตัวแปรจาก string เป็น integer เป็นต้น ตัวภาษาใน Visual Studio 6.0 จะให้เราทำได้ แต่ใน .NET จะมีการตรวจสอบว่าถ้ามีการเปลี่ยนแปลงค่าจาก sting เป็น integer หรือการเปลี่ยนชนิดของตัวแปรอื่นๆที่จะทำให้เกิด Runtime Error นั้นตัว Compiler จะตรวจสอบได้และก็จะมีการแจ้งข้อผิดพลาดให้ทราบ

แม้แต่เรื่องของตัวแปรที่เอาไปใช้งาน ถ้าเราประกาศตัวแปรขึ้นมาแล้วเราเอาไปใช้งานโดยไม่ได้มีการกำหนดค่าให้มันก่อน Compiler ก็จะคอยดัก นั่นคือ Compiler มีความฉลาดมากขึ้น เพื่อดักข้อผิดพลาด ณ จุดที่คอมไพล์เลย ซึ่งดีกว่าให้เกิด Runtime Error ขึ้นทำให้เราแก้ไขข้อผิดพลาดได้ยาก หรือแม้แต่เรื่องอาเรย์ก็ตาม ถ้าเราประกาศอาเรย์ไว้ 100 ไอเทมถ้าบรรทัดใดในไค้ดของเราเกิดมีการอ้างถึงอาเรย์ขึ้นมา Compiler ก็จะคอมไพล์ไม่ผ่านทันที ซึ่งจะกำหนดไว้ว่าตรงนี้เกิด Runtime Error ได้ตอนที่รัน นี่ก็ความฉลาดมากยิ่งขึ้นในตัว Compiler ของ Visual Studio.net ซึ่งจะทำให้เราพัฒนาแอปพลิเคชันได้สะดวก โดยปราศจาก Runtime Error มากที่สุด

ในเรื่องระบบรักษาความปลอดภัยนั้น ตัว .NET ก็สามารถกำหนดได้ว่าผู้ที่จะรัน แอปพลิเคชันของเรานั้นมีสถานะ (permission) เป็นอะไร อาจจะต้องการสถานะเป็น admin ถึงจะรัน แอปพลิเคชันของเราได้

2.6.2.2.5 รูปแบบการทำงานร่วมกับภาษาอื่นๆ

การพัฒนาแอปพลิเคชันใน .NET รวมไปถึงเทคนิคต่างๆ ที่กล่าวมาแล้วนั้น ไม่จำเป็นจะใช้ เฉพาะภาษาตระกูลใน Visual Studio ไม่ว่าจะเป็น VB.net, C# หรือ C++ ถ้าภาษาใดก็ตามสามารถที่จะ คอมไพล์มาให้เป็น Common Language Runtime ได้ ภาษานั้นก็จะสามารถใช้คุณลักษณะทั้งหมดของ .NET ได้ ซึ่งตอนนี้มีภาษ่อื่นๆ ที่สามารถทำงานร่วมกับแอปพลิเคชัน .NET มากกว่า 20 ภาษาที่ พัฒนาตัว Compiler ขึ้นมาเพื่อคอมไพล์ภาษาต่างๆ ให้อยู่ในรูปแบบของแพลตฟอร์ม .NET ไม่ว่าจะเป็น ภาษา COBOL, PASCAL และอื่นๆ

ในตัว Visual Studio.net ก็มีเครื่องมือระดับสูง (High Level Tools) ที่จะทำให้เราพัฒนา แอปพลิเคชันได้สะดวกและง่ายมากขึ้น นี่คือเรื่องของ Common Language Runtime ซึ่งถือได้ว่าเป็น เลเยอร์ล่างสุดของการคอมไพล์แอปพลิเคชันที่เขียนด้วย Visual Studio.net

2.6.2.3 เลเยอร์ Base Class Library

เลเยอร์ถัดมาของโครงสร้างการพัฒนาแอปพลิเคชัน .NET ต่อจาก Common Language Runtime ก็คือ Base Class Library เทคนิคต่างๆ ที่ Visual Studio.net จัดเตรียมให้เราในการใช้งานนั้น Base Class Library เปรียบเสมือน เป็นการรวบรวมเอาฟังก์ชันของ API (Application Programming Interface) ทั้งหมด สมมติว่าตอนที่เรากำลังพัฒนาแอปพลิเคชันด้วย Visual Studio 6.0 เวลาเราจะเรียกใช้ งานฟังก์ชันบางอย่างในระดับสูง หรือต้องการทำงานแบบลึกๆกับระบบ เรามักจะเรียกใช้ API นี้ภาษา Visual Studio 6.0 มีความสามารถในการเรียกฟังก์ชัน API ได้ต่างกัน API บางตัวในปัจจุบันอาจจะ ต้องใช้ C++ ถึงจะเรียกได้ เพราะว่าต้องใช้โครงสร้างข้อมูล (Data Structure) หรือพอยเตอร์ อะไร บางอย่างที่ไม่ใช่ใน Visual Basic แต่ API บางตัวก็สามารถเรียกจาก Visual Basic ได้

ตัว Base Class Library ก็คือ การที่เรารวบรวมฟังก์ชัน API ซึ่งกระจัดกระจายอยู่ เวลาจะ เรียกใช้เราต้องไปค้นหาใน Help นั่นคือ Base Class Library พยายามที่จะรวบรวม API และฟังก์ชัน ทั้งหมดเกี่ยวกับระบบเข้ามาไว้ในลักษณะของ Object Oriented ทำเป็นคลาสอันหนึ่งซึ่งเป็นมาตรฐาน เป็นคลาสที่สร้างมาในตัวระบบเรียบร้อยแล้ว ซึ่งคลาสทั้งหมดจะอยู่ภายใต้คลาสหลักอันหนึ่งที่เรียกว่า System

ทุกอย่างที่พัฒนาด้วยภาษาใน Visual Studio.net จะเป็น Object Oriented ทั้งหมด โดยมีคลาสที่ใหญ่ที่สุดเรียกว่าคลาส System ซึ่งภายใต้คลาส System จะมีคลาสย่อยๆมากมาย ซึ่งแต่ละคลาสจะสนับสนุนฟังก์ชัน API หรือสนับสนุนการทำงานที่เราต้องได้ไม่ว่าจะเป็นเรื่องของการทำกราฟฟิก การทำเกี่ยวกับโครงสร้างข้อมูล (data structure) การทำเกี่ยวกับเรื่องเครือข่าย (network) ฟังก์ชัน API เหล่านี้จะถูกจัดกลุ่มให้เป็น Object Oriented อยู่ภายใต้ System Class การเรียกใช้งาน System Class ถ้าเป็น VB กับ C# ก็ใช้งานได้ทั้ง 2 อย่าง

เลขอร์ที่ถูกส่งขึ้นมาจาก Base Class Library รวมทั้งเป็นแนวคิดซึ่งไม่โครซอฟท์ผลักดันมาก คือเรื่องของ Web Service นั่นเอง

2.6.2.4 เลขอร์ Common Language Specification

เลขอร์สุดท้ายในสถาปัตยกรรม .NET ที่เราจะพูดถึงก็คือ Common Language Specification คือเรื่องของมาตรฐานภาษาบนพื้นฐาน .NET ซึ่ง Compiler จะต้อง ทำงานตามมาตรฐานดังกล่าว เพื่อให้สามารถทำงานร่วมกับภาษาพื้นฐาน .NET และภาษาอื่นๆได้ ไมโครซอฟท์ได้ทำการปรับภาษาต่างๆ เช่น C#,VB ให้เข้ามาตรฐาน .NET นอกจากนั้น ผู้ผลิตรายอื่นสามารถพัฒนาตามข้อกำหนดนี้ เพื่อให้สามารถทำงานบนพื้นฐาน .NET ได้

เลขอร์ที่ 3 ของสถาปัตยกรรม .NET ก็คือเครื่องมือที่ใช้ในการสร้างแอปพลิเคชัน หรือ หลักการที่ใช้ในการเขียนโปรแกรมต่างๆ เช่นเรื่องของ ADO.net, ASP.net ที่ใช้ในการพัฒนาแอปพลิเคชัน แต่สิ่งที่อยู่เหนือกว่าทุกอย่างก็คือภาษาที่เราใช้งาน ภาษาต่างๆที่ทำงานใน .NET นั้นมีข้อดีคือ ต้องสนับสนุนมาตรฐานเดียวกัน เรียกว่า Common Language Specification ซึ่งไมโครซอฟท์ได้จดทะเบียนมาตรฐานนี้เข้ากับองค์กร ECMA แล้ว ซึ่งเป็นองค์กรที่ทำงานเกี่ยวกับการสร้างมาตรฐานกลางของระบบสื่อสารและข้อมูลคอมพิวเตอร์ อีกทั้งยังเป็นแบบเปิด (open) ด้วย เพราะฉะนั้นเจ้าของภาษาอื่นๆ ก็สามารถสร้างตัวแปลภาษา หรือ Compiler เพื่อคอมไพล์ภาษาของเขาให้เข้ามาเป็น Common Language Specification อันนี้ได้ ในไม่ช้าเราจะเห็นเว็บเพจเขียนด้วยภาษา COBOL ก็ได้ รวมทั้งภาษาอื่นๆด้วย นอกจากนี้ในตระกูล .NET เองเราก็มี Visual Basic, Visual C++ กับ Visual C# ซึ่งภาษาอื่นๆ ที่เรากล่าวถึงนั้นปัจจุบันก็มีกว่า 20 ภาษา เช่น ภาษา MEL, COBOL,PASCAL,ไอเฟล , ไพทอน, Perl, Smalltalk ภาษา ML ซึ่งเป็นภาษาที่ใช้งานกันในต่างประเทศ ภาษาประเภท Object ทั้งหมดสามารถทำเป็นแพลตฟอร์มของ .NET ได้เพราะว่าใน .NET นั้นผลิตทุกอย่างเป็น Object Oriented ทั้งหมด

2.6.3 ภาษาและเครื่องมือของ Microsoft.NET

Microsoft.Net ใช้ภาษา C# เป็นหลักในการพัฒนาแอปพลิเคชัน C# พัฒนามาจากภาษา C และ C++ โดย C# นำแนวคิดเรื่อง Garbage collection และ Hierarchical namespace ของ Java มาใช้ อย่างไรก็ตามการจัดการ Exception ของ C# ก็มีข้อยกเว้นให้โปรแกรมเมอร์สามารถละเว้น Exception ได้ในบางกรณีที่ต้องการ ต่างกับใน Java ที่การจัดการ Exception เป็นไปอย่างเคร่งครัด ดังนั้นโอกาสที่จะเกิดข้อผิดพลาดในการเขียนโปรแกรมจึงไม่มีโอกาสเกิดได้เลย นอกจากนี้ใน C# ยังสร้างใช้ตัวแปรพอยน์เตอร์ได้เหมือนในภาษา C โปรแกรมเมอร์ที่เคยใช้พอยน์เตอร์ส่วนใหญ่ทราบดีว่ามีโอกาสพลาดได้ง่าย และผลเสียที่เกิดจากการเขียนโปรแกรมโดยใช้พอยน์เตอร์ไม่รอบคอบนั้นมีมาก การตรวจสอบก็ทำได้ยาก Java ด้ปัญหาเรื่องนี้ไปเลย เพราะไม่อนุญาตให้ใช้พอยน์เตอร์ อย่างไรก็ตามมีคนแย้งว่าการที่ C# เปิดช่องให้จัดการ Exception เองนั้นทำให้เกิดความอิสระในการเขียนโปรแกรม และการใช้พอยน์เตอร์ก็เป็นประโยชน์ในการเขียนโปรแกรม และบางครั้งก็จำเป็นต้องใช้ สำหรับเรื่องภาษานี้ก็ต้องแล้วแต่ความถนัดของแต่ละคน ใครที่ชอบภาษาไหนก็อ้างเหตุผลมาเข้าข้างตัวเองได้ทั้งนั้น เพียงแต่โอกาสพลาดจากการใช้ภาษา C# มีมากกว่าถ้าไม่ระมัดระวัง

Microsoft.Net สนับสนุนการใช้ภาษาอื่นในการพัฒนาแอปพลิเคชัน ผู้ใช้สามารถเขียนโปรแกรมด้วย Visual Basic, COBOL, FORTRAN ได้ เพราะตัวคอมไพเลอร์จะแปลงออกมาเป็นไบท์โค้ด Internal Language ซึ่งโค้ดที่ได้จะทำงานบน CLR ตามที่กล่าวไปแล้ว เพียงแต่ว่าทุกอย่างที่พูดถึงนี้ต้องอยู่บนวินโดวส์เท่านั้น

เครื่องมือช่วยพัฒนาแอปพลิเคชันของ Microsoft.Net ก็คือ Visual Studio.Net ที่ทุกคนต้องยอมรับว่าเป็น IDE ที่มีประสิทธิภาพ เพราะใช้งานง่ายและประสานการทำงานได้ดี

2.6.4 พอร์ตเทบิลิตี้ของระบบ

พิจารณา Microsoft.Net เนื่องจาก .Net ทำงานบนวินโดวส์ ดังนั้นจึงไม่มีปัญหาเรื่องการสนับสนุนฮาร์ดแวร์ แต่การทำงานบนแพลตฟอร์มอื่น ปัจจุบันยังเป็นไปไม่ได้ แม้ว่าไมโครซอฟท์บอกว่าจะพยายามพัฒนา COM เพื่อนำไปใช้บนระบบปฏิบัติการอื่นก็ตาม

อย่างไรก็ตามสิ่งสำคัญที่ต้องพิจารณาเรื่องพอร์ตเทบิลิตี้ก็คือ แพลตฟอร์ม หากงานที่ทำนั้นมีโอกาสเกี่ยวข้องกับลูกค้าที่ใช้งานแพลตฟอร์มหลายแบบ ก็ไม่ควรที่จะใช้ .Net

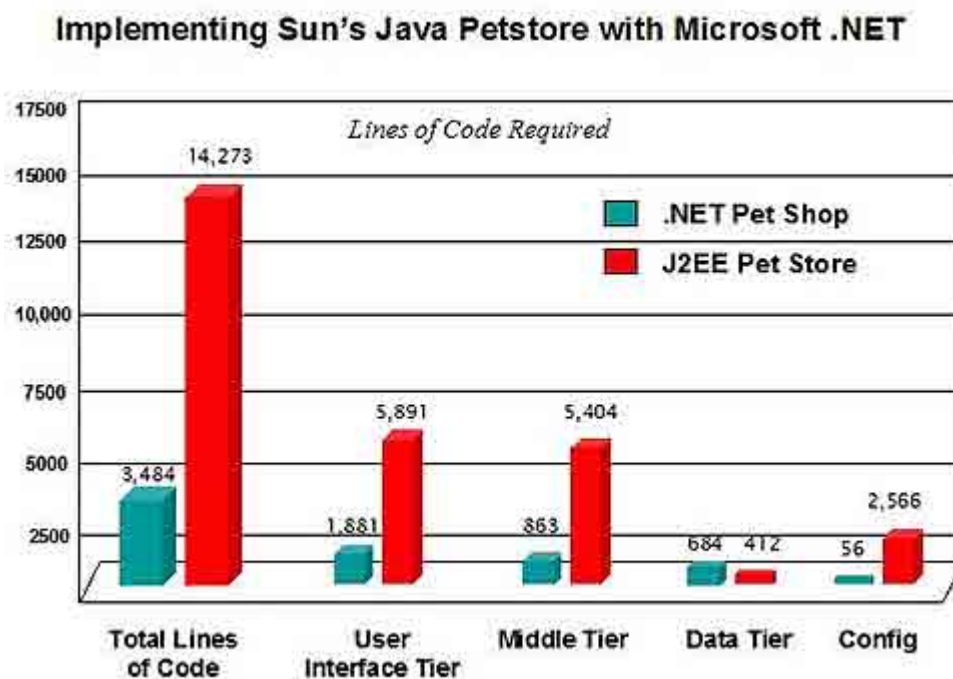
2.6.5 การเปรียบเทียบระหว่าง Microsoft.Net กับ J2EE

2.6.5.1 การสนับสนุนเว็บเซอร์วิส

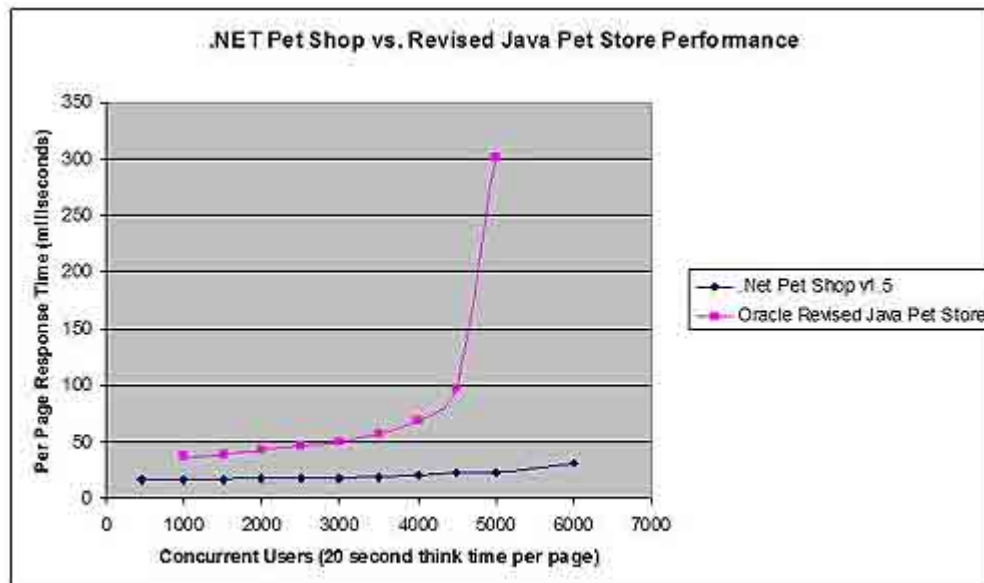
ทั้ง J2EE และ Microsoft.Net ก็มีความสามารถด้านเว็บเซอร์วิสเหมือนกัน จาวามีชุดเอพีไอที่ใช้จัดการ XML ได้ โดยตรง สนับสนุนการใช้งานร่วมกับ SOAP, UDDI และ WSDL แต่งานส่วนใหญ่ยังต้องเขียนขึ้นเอง ซึ่งเป็นงานที่ต้องใช้เวลาพอสมควร ในขณะที่การพัฒนาเว็บเซอร์วิสบน Microsoft.Net นั้นมีเครื่องมือหลายอย่างที่จะช่วยในการทำงาน ลดเวลาที่ต้องใช้ในการทำงานและทำได้ง่ายมากกว่า J2EE พอสมควร

2.6.5.2 ประสิทธิภาพการทำงาน

มีข้อมูลแสดงการทดสอบประสิทธิภาพของ Microsoft.Net เปรียบเทียบกับ Sun J2EE ที่แสดงอยู่ที่ <http://www.gotdotnet.com/team/compare> โดยใช้โซลูชันจาก Oracle, IBM และ Sun พัฒนาแอปพลิเคชัน (หากผู้อ่านสนใจขอให้อ่านรายละเอียดได้จากเว็บไซต์ข้างต้น) จากรูปที่ 1 และ 2 แสดงผลการทดสอบที่ ปรากฏว่า Microsoft.Net ทำผลงานได้น่าประทับใจ ทั้งห่าง J2EE ในหลายหัวข้อ ดังที่แสดงในรูปที่ 2.9 และ รูปที่ 2.10



รูปที่ 2.9 Implementing Sun's Java Petstore with Microsoft .Net



รูปที่ 2.10 .NET Pet Shop vs. Revised Java Pet Performance